

Why Enum Singleton are better in Java

Enum Singletons are new way to implement Singleton [pattern](#) in Java by using Enum with just one instance. Though Singleton pattern in Java exists from long time Enum Singletons are relatively new concept and in practice from Java 5 onwards after introduction of Enum as keyword and feature. This article is somewhat related to my earlier post on Singleton, [10 interview questions on Singleton pattern in Java](#) where we have discussed common questions asked on interviews about Singleton pattern and [10 Java enum examples](#), where we have seen how versatile enum can be. This post is about **why should we use Enum as Singleton in Java**, What benefit it offers compared to conventional singleton methods etc.

Java Enum and Singleton Pattern



Following are some reasons which make sense to me for using Enum to implement Singleton pattern in Java. By the way if you like articles on design pattern than you can also [check](#) my post on [Builder design pattern](#) and [Decorator design pattern](#).

1) Enum Singletons are easy to write

This is by far biggest advantage, if you have been writing Singletons prior to Java 5 then you know that even with double checked locking you can have more than one instances. though that issue is fixed with Java memory model improvement and [guarantee](#) provided by volatile variables from Java 5 onwards but it still tricky to write for many beginners. compared to double checked locking with synchronization Enum singletons are cake [walk](#). If you don't believe than just compare below [code for](#) conventional singleton with double checked locking and Enum Singletons:

Singleton using Enum in Java

[This is the way](#) we generally declare Enum Singleton, it may contain instance variable and instance method but for sake of simplicity I haven't used any, just beware that if you are using any instance method than you need to ensure thread-safety of that method if at all it affect the state of object. By default creation of Enum instance is thread safe but any other method on Enum is programmers responsibility.

```
/**
 * Singleton pattern example using Java Enum
 */
public enum EasySingleton{
    INSTANCE;
}
```

You can access it by `EasySingleton.INSTANCE`, much easier than calling `getInstance()` method on Singleton.

Singleton example with double checked locking

Below code is an example of double checked locking in Singleton pattern, here `getInstance()` method checks two times to see whether `INSTANCE` is null or not and that's why it's called double checked locking pattern, remember that double checked locking is [broken](#) before Java 5 but with the guarantee of [volatile variable in Java 5](#) memory model, it should work perfectly.

```
/**
 * Singleton pattern example with Double checked Locking
 */
public class DoubleCheckedLockingSingleton{
    private volatile DoubleCheckedLockingSingleton INSTANCE;

    private DoubleCheckedLockingSingleton(){

    }

    public DoubleCheckedLockingSingleton getInstance(){
        if(INSTANCE == null){
            synchronized(DoubleCheckedLockingSingleton.class){
                //double checking Singleton instance
                if(INSTANCE == null){
                    INSTANCE = new DoubleCheckedLockingSingleton();
                }
            }
        }
        return INSTANCE;
    }
}
```

You can call `DoubleCheckedLockingSingleton.getInstance()` to get access of this Singleton class.

Now Just look at amount of code needed to create a **lazy loaded thread-safe Singleton**. With Enum Singleton pattern you can have that in one line because creation of Enum instance is [thread-safe](#) and guaranteed by JVM.

People may argue that there are better way to write Singleton instead of Double checked locking approach but every approach has there own advantages and disadvantages like I mostly prefer static field Singleton initialized during classloading as shwon in below example, but keep in mind that is not a **lazy loaded Singleton**:

Singleton pattern with static factory method

This is one of my favorite method to implemnt Singleton pattern in Java, Since Singleton instance is [static](#) and [final variable](#) it initialized when class is first loaded into memeory so creation of instance is inherently thread-safe.

```
/**
 * Singleton pattern example with static factory method
 */

public class Singleton{
    //initailzed during class loading
    private static final Singleton INSTANCE = new Singleton();

    //to prevent creating another instance of Singleton
    private Singleton(){}

    public static Singleton getSingleton(){
        return INSTANCE;
    }
}
```

You can call `Singleton.getSingleton()` to get access of this class.

2) Enum Singletons handled Serialization by themselves

Another problem with conventional Singletons are that once you implement [serializable interface](#) they are no longer remain Singleton because `readObject()` method always return a new instance just like constructor in Java. you can avoid that by using `readResolve()` method and discarding newly created instance by replacing with Singeton as shwon in below example :

```
//readResolve to prevent another instance of Singleton
private Object readResolve(){
    return INSTANCE;
}
```

This can become even more complex if your Singleton Class maintain state, as you need to make them [transient](#), but withn **Enum Singleton**, Serialization is guarnateed by JVM.

3) Creation of Enum instance is thread-safe

As stated in point 1 since creatino of Enum instance is thread-safe by default you don't need to worry about double checked locking.

In [summary](#), given the **Serialization and thraead-safety guaranteed** and with couple of line of code enum Singleton pattern is best way to create Singleton in Java 5 world. you can still use other popular methods if you feel so but I still have to find a *convincing reason not to use Enum as Singleton*, let me know if you got any.